

Bachelor-Thesis

zur Erlangung des akademischen Grades

Bachelor of Science (B. Sc.)

an der Hochschule für Technik und Wirtschaft des Saarlandes

im Studiengang Praktische Informatik

der Fakultät für Ingenieurwissenschaften

Netzwerk-Monitoring Dashboard mit Automatisierung

vorgelegt von

Xudong Zhang

betreut und begutachtet von

Prof. Dr.-Ing. André Miede

Andreas Schaffhauser

Saarbrücken, den 16. August 2026

Selbständigkeitserklärung

Ich versichere, dass ich die vorliegende Arbeit (bei einer Gruppenarbeit: den entsprechend gekennzeichneten Anteil der Arbeit) selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Ich erkläre hiermit weiterhin, dass die vorgelegte Arbeit zuvor weder von mir noch von einer anderen Person an dieser oder einer anderen Hochschule eingereicht wurde.

Darüber hinaus ist mir bekannt, dass die Unrichtigkeit dieser Erklärung eine Benotung der Arbeit mit der Note „nicht ausreichend“ zur Folge hat und einen Ausschluss von der Erbringung weiterer Prüfungsleistungen zur Folge haben kann.

Saarbrücken, den 16. August 2026

Xudong Zhang

Zusammenfassung

Kurze Zusammenfassung des Inhaltes in deutscher Sprache, der Umfang beträgt zwischen einer halben und einer ganzen DIN A4-Seite.

Orientieren Sie sich bei der Aufteilung bzw. dem Inhalt Ihrer Zusammenfassung an Kent Becks Artikel: <http://plg.uwaterloo.ca/~migod/research/beck00PSLA.html>.

*We have seen that computer programming is an art,
because it applies accumulated knowledge to the world,
because it requires skill and ingenuity, and especially
because it produces objects of beauty.*

— Donald E. Knuth [5]

Danksagung

Hier können Sie Personen danken, die zum Erfolg der Arbeit beigetragen haben, beispielsweise Ihren Betreuern in der Firma, Ihren Professoren/Dozenten an der htw saar, Freunden, Familie usw.

Inhaltsverzeichnis

1	Einleitung	1
1.1	Motivation und Zielsetzung	1
1.2	Problemstellung	2
1.3	Ziele der Arbeit	2
1.3.1	Pflichtziele	3
1.3.2	Optionale Erweiterungen	4
1.4	Geplantes Vorgehen	4
1.5	Aufbau der Arbeit	5
2	Grundlage	7
2.1	Grundbegriffe und Definitionen	7
2.1.1	Netzwerk-Monitoring, Verfügbarkeit und Latenz	7
2.1.2	Metrikbasiertes Monitoring und Alerting	7
2.1.3	Containerisierung und TCP-basierte Erreichbarkeitsprüfung	8
2.2	Technologische Grundlagen	8
2.2.1	Spring Boot, Actuator und Micrometer	8
2.2.2	Prometheus, Grafana und Alertmanager	9
2.2.3	Containerisierte Bereitstellung mit Docker und Docker Compose	9
2.3	Verwendete Methoden und Konzepte des entwickelten Systems	9
2.3.1	Konfigurationsgetriebenes Gerätemanagement und zeitgesteuertes Polling	10
2.3.2	Zustandsbasierte Metriken und Kennzahlenmodell	10
2.3.3	Schichtenarchitektur, Fehlerbehandlung und Robustheit	10
2.3.4	Automatisierte Tests der Kernlogik	11
2.4	Stand der Technik und Einordnung	11
3	Analyse	13
4	Konzeption	15
5	Implementierung	17
6	Evaluation	19
7	Zusammenfassung und Ausblick	21
	Literatur	23
	Abbildungsverzeichnis	25
	Tabellenverzeichnis	25
	Listings	25
	Abkürzungsverzeichnis	27

1 Einleitung

1.1 Motivation und Zielsetzung

Die fortschreitende Digitalisierung betrieblicher Abläufe führt dazu, dass die Verfügbarkeit von Netzwerkdiensten und Anwendungen in modernen IT-Umgebungen eine zentrale Voraussetzung für einen stabilen Arbeitsbetrieb darstellt. Bereits kleinere Störungen in Netzwerkverbindungen oder auf Anwendungsebene können dazu führen, dass Arbeitsprozesse unterbrochen, Reaktionszeiten verlängert oder zentrale IT-Dienste nur eingeschränkt nutzbar werden. Vor diesem Hintergrund gewinnt ein systematisches und möglichst automatisiertes Monitoring zunehmend an Bedeutung.

Die Wahl des Themas „Netzwerk-Monitoring Dashboard mit Automatisierung“ ergibt sich aus mehreren fachlichen und praktischen Aspekten.

- **Praxisbezug aus dem Praktikum:** Im Rahmen des IT-Service-Praktikums beim Bildungscampus Saarland wurde deutlich, wie stark Netzwerkstörungen den laufenden Betrieb beeinträchtigen können. Dabei zeigte sich zugleich die Relevanz eines modernen und automatisierten Netzwerk-Monitorings.
- **Berufliche Relevanz:** Die angestrebte fachliche Ausrichtung liegt im Bereich DevOps und Netzwerkinfrastruktur. Monitoring stellt in diesem Umfeld eine zentrale Kompetenz dar und besitzt zugleich eine hohe Bedeutung auf dem aktuellen IT-Arbeitsmarkt.
- **Technisches Interesse:** Das Thema verbindet zentrale Interessensfelder der praktischen Informatik, insbesondere Netzwerkinfrastruktur, Automatisierung sowie moderne Werkzeuge des IT-Betriebs.

Der Praxisbezug des Themas ist besonders relevant, da Monitoring im IT-Service nicht nur der technischen Beobachtung von Systemen dient, sondern unmittelbar zur Sicherstellung betrieblicher Abläufe beiträgt. Werden Netzwerkprobleme oder Leistungsabfälle zu spät erkannt, können Helpdesk-Prozesse, Fernzugriffe, interne Verwaltungsanwendungen oder weitere digitale Dienste verzögert oder gestört werden. Ein automatisiertes Monitoring kann in solchen Situationen dazu beitragen, Störungen früher zu erkennen, Zustände transparenter darzustellen und die Reaktionsfähigkeit des IT-Betriebs zu verbessern.

Darüber hinaus besitzt das Thema auch aus fachlicher Sicht eine hohe Relevanz, da es mehrere Kernbereiche der praktischen Informatik miteinander verbindet. Dazu zählen unter anderem verteilte Systeme, Netzwerkinfrastruktur, automatisierte Datenerfassung, Zeitreihenverarbeitung, Visualisierung und einfache Alarmierungsmechanismen. Die Arbeit greift damit sowohl technische als auch organisatorische Fragestellungen auf und eignet sich daher besonders für eine praxisnahe Abschlussarbeit mit wissenschaftlichem Anspruch.

Das übergeordnete Ziel der Arbeit besteht darin, ein nachvollziehbares und technisch realisierbares Konzept für ein Monitoring-System zu entwickeln und dessen Umsetzbarkeit anhand eines Prototyps zu demonstrieren. Neben der rein technischen Realisierung soll auch gezeigt werden, wie sich verschiedene Werkzeuge sinnvoll kombinieren lassen,

1 Einleitung

um einen konsistenten Monitoring-Prozess von der Datenerfassung bis zur Visualisierung und Alarmierung abzubilden.

1.2 Problemstellung

In modernen IT-Infrastrukturen ist die kontinuierliche Überwachung von Netzwerkverbindungen, insbesondere im LAN- und VPN-Bereich, sowie des Zustands von Anwendungen von zentraler Bedeutung für einen stabilen und zuverlässigen Betrieb. Gerade in IT-Service-Umgebungen können Netzwerkstörungen oder Fehler auf Anwendungsebene in kurzer Zeit zu erheblichen betrieblichen Beeinträchtigungen führen. Dies gilt insbesondere dann, wenn mehrere Systeme voneinander abhängen und Störungen nicht isoliert, sondern kettenartig auf andere Dienste wirken.

In vielen IT-Service-Abteilungen werden Netzwerkprobleme jedoch weiterhin vor allem durch manuelle Ping-Tests oder unregelmäßige Einzelprüfungen erkannt. Solche Verfahren können zwar in einfachen Situationen kurzfristig hilfreich sein, sie sind jedoch nur bedingt geeignet, um größere oder dynamische Infrastrukturen zuverlässig zu überwachen. Die fehlende Automatisierung hat zur Folge, dass Störungen verspätet entdeckt werden, der manuelle Prüfaufwand hoch bleibt, historische Daten für weiterführende Analysen fehlen und bei Ausfällen keine unmittelbare Alarmierung erfolgt.

Diese Vorgehensweise ist nicht nur ineffizient, sondern auch fehleranfällig. Manuelle Kontrollen sind stark vom Zeitpunkt der Durchführung, vom Erfahrungswissen der Beteiligten und von der systematischen Dokumentation einzelner Beobachtungen abhängig. Dadurch entsteht das Risiko, dass kurzzeitige Ausfälle unbemerkt bleiben oder dass Zusammenhänge zwischen verschiedenen Fehlerbildern im Nachhinein nicht mehr zuverlässig rekonstruiert werden können.

Hinzu kommt, dass ohne kontinuierlich gespeicherte Messwerte nur eingeschränkt beurteilt werden kann, ob ein Problem einmalig aufgetreten ist oder ob sich bereits über längere Zeit ein negativer Trend abgezeichnet hat. Für einen professionellen IT-Betrieb ist jedoch nicht nur die unmittelbare Störungserkennung relevant, sondern auch die Möglichkeit, Entwicklungen über einen längeren Zeitraum nachzuvollziehen, Grenzwerte zu definieren und daraus gezielte Maßnahmen abzuleiten.

Ein professioneller IT-Betrieb erfordert daher ein Monitoring, das automatisiert, kontinuierlich und nachvollziehbar arbeitet. Es sollte Metriken auf unterschiedlichen Ebenen erfassen, diese zentral speichern, für operative und analytische Zwecke visualisieren und bei kritischen Zuständen eine zeitnahe Alarmierung ermöglichen. Genau an dieser Stelle setzt die vorliegende Arbeit an, indem ein prototypisches Monitoring-System für Netzwerk- und Anwendungsebene konzipiert und umgesetzt wird.

1.3 Ziele der Arbeit

Ziel der Arbeit ist die Entwicklung eines Monitoringsystems für LAN-/VPN-Verbindungen sowie für eine Spring-Boot-Anwendung. Das System soll Messdaten automatisiert erfassen, speichern, visualisieren und bei kritischen Zuständen eine Alarmierung auslösen. Damit verfolgt die Arbeit das Ziel, einen durchgängigen technischen Ablauf von der Generierung über die Erfassung und Speicherung bis hin zur Auswertung und Benachrichtigung abzubilden.

Die wesentlichen Zielsetzungen lauten wie folgt:

- Erfassung von Anwendungsmetriken durch eine Spring-Boot-Komponente mit Micrometer, insbesondere HTTP-Antwortzeiten, Fehlerquoten, Thread-Anzahl, JVM-

Speicher und CPU-Auslastung.

- Überwachung zusätzlicher Netzwerkmetriken wie Ping-Latenz, Erreichbarkeit, VPN-Status und Port-Verfügbarkeit ausgewählter Dienste.
- Regelmäßige Speicherung aller Metriken in Prometheus als Zeitreihendatenbasis zur Auswertung aktueller und historischer Werte.
- Visualisierung zentraler Kennzahlen aus Anwendung, Netzwerk und Host-Systemen in einem Grafana-Dashboard.
- Umsetzung einer einfachen Alarmierungsfunktion, die bei Ausfällen oder Schwellenwertüberschreitungen automatisiert eine E-Mail-Benachrichtigung versendet.

Die einzelnen Zielsetzungen greifen ineinander. Die Datenerfassung bildet die Grundlage für jede weitere Verarbeitung, da ohne verlässliche Metriken weder eine Zeitreihenspeicherung noch eine aussagekräftige Visualisierung möglich ist. Ebenso setzt eine funktionierende Alarmierung voraus, dass Zustandsänderungen und Grenzwertverletzungen zunächst systematisch erkannt und ausgewertet werden.

Aus fachlicher Sicht verfolgt die Arbeit damit nicht nur die Implementierung einzelner Werkzeuge, sondern die Entwicklung einer durchgängigen Monitoring-Kette. Entscheidend ist dabei, dass die verschiedenen Komponenten nicht isoliert betrachtet, sondern als zusammenhängendes System verstanden werden. Dadurch wird die Arbeit über eine reine Tool-Installation hinaus erweitert und erhält einen eigenständigen konzeptionellen Mehrwert.

Um den Arbeitsumfang innerhalb des vorgegebenen Bearbeitungszeitraums realistisch zu gestalten, wird zwischen Pflichtzielen und optionalen Erweiterungen unterschieden. Diese Trennung ist notwendig, um einen klaren Mindestumfang festzulegen und zugleich weiterführende Entwicklungsmöglichkeiten transparent zu machen.

1.3.1 Pflichtziele

Die Pflichtziele definieren den Kern der Arbeit und müssen im Rahmen des Projekts umgesetzt werden. Sie bilden zugleich die Grundlage für die spätere Bewertung des Prototyps.

- **Datenerfassung:** Entwicklung einer Spring-Boot-API mit Micrometer-Anbindung zur Erfassung von JVM-Metriken wie Speicher, CPU-Auslastung und Threads sowie von HTTP-Metriken wie Antwortzeit und Fehlerquote.
- **Speicherung:** Aufbau einer Prometheus-basierten Zeitreihenspeicherung mit definierten Scraping-Intervallen, beispielsweise 15 Sekunden, um aktuelle und historische Daten auswertbar zu machen.
- **Visualisierung:** Erstellung eines Grafana-Dashboards mit Panels für Ping-Latenz, VPN-Uptime, Port-Status und JVM-Leistung.
- **Alarmierung:** Implementierung eines Bash-Skripts, das bei Schwellenwertüberschreitungen, beispielsweise bei einer Ping-Latenz von über 200 ms oder einem VPN-Ausfall, automatisiert eine E-Mail-Benachrichtigung versendet.

Diese Pflichtziele decken die für das Vorhaben zentralen Funktionsebenen ab. Sie stellen sicher, dass sowohl auf Netzwerk- als auch auf Anwendungsebene relevante Zustände beobachtet werden können und dass die erhobenen Informationen nicht nur gesammelt,

1 Einleitung

sondern auch praktisch nutzbar gemacht werden. Die Kombination aus Datenerhebung, Speicherung, Visualisierung und Alarmierung bildet damit den funktionalen Kern des Prototyps.

1.3.2 Optionale Erweiterungen

Die optionalen Erweiterungen gehen über den definierten Mindestumfang hinaus und dienen dazu, mögliche Ausbaustufen des Systems aufzuzeigen. Sie sind insbesondere dann relevant, wenn einzelne Kernfunktionen früher als geplant umgesetzt werden oder wenn sich im Projektverlauf zusätzlicher Spielraum ergibt.

- **Ansible-Integration:** Automatisierte Bereitstellung der Monitoring-Infrastruktur auf mehreren Servern.
- **Zusätzliche Dashboards:** Getrennte Ansichten für Anwendungs-, Netzwerk- und Host-Ebene.
- **Erweiterte Alarmregeln:** Mehrkriterielle Alarmierung, etwa bei gleichzeitiger Prüfung von VPN-Status, Ping-Latenz und Port-Erreichbarkeit.
- **Rollen- und Rechte management:** Verwaltung von Benutzerrollen und Zugriffsrechten innerhalb des Grafana-Dashboards.

Die optionalen Erweiterungen verdeutlichen, dass das entworfene System grundsätzlich skalierbar und ausbaufähig angelegt ist. Gleichzeitig unterstreichen sie, dass die vorliegende Arbeit einen bewussten Fokus auf einen belastbaren Kernumfang legt, ohne weiterführende Entwicklungsmöglichkeiten auszuschließen.

1.4 Geplantes Vorgehen

Das Vorgehen orientiert sich an einem typischen Ablauf informatischer Abschlussarbeiten. Zunächst wird der fachliche und organisatorische Rahmen präzisiert, insbesondere im Hinblick auf die zu überwachenden Hosts, Netzwerkpfade und VPN-Verbindungen, geeignete Schwellenwerte sowie datenschutzbezogene Anforderungen. Diese Präzisierung ist notwendig, um ein realistisches und konsistentes Zielsystem für die weiteren Projektphasen festzulegen.

Anschließend erfolgt eine Recherche zu Spring Boot, Micrometer, Prometheus und Grafana, um die Eignung dieser Technologien für das geplante Monitoring-Szenario zu bewerten. Dabei geht es nicht nur um eine funktionale Betrachtung einzelner Werkzeuge, sondern auch um deren Zusammenspiel innerhalb einer Gesamtarchitektur. Die Recherche dient damit zugleich als Grundlage für spätere Technologieentscheidungen.

Darauf aufbauend wird ein Systementwurf entwickelt, der Architektur und Datenflüsse der Lösung beschreibt. Dieser Entwurf legt fest, welche Komponenten miteinander kommunizieren, wie Metriken erzeugt und übertragen werden und an welchen Stellen Visualisierung sowie Alarmierung ansetzen. Eine saubere konzeptionelle Vorarbeit ist wesentlich, damit die spätere Implementierung nicht nur funktioniert, sondern auch nachvollziehbar strukturiert ist.

Danach erfolgt die prototypische Implementierung, vorzugsweise in einer containerisierten Umgebung mit Docker. Die Containerisierung erleichtert eine reproduzierbare Bereitstellung der einzelnen Komponenten und unterstützt damit sowohl die technische

Umsetzung als auch die spätere Demonstration des Systems. In diesem Schritt wird geprüft, ob sich die zuvor entworfene Architektur mit den ausgewählten Technologien konsistent umsetzen lässt.

Abschließend wird das System anhand typischer Fehlerszenarien wie erhöhter Ping-Latenz, geschlossenen Ports oder eines VPN-Ausfalls getestet und hinsichtlich Erkennungszeit, Dashboard-Qualität und Alarmierungsfunktion bewertet. Die Evaluation dient nicht nur dazu, die Funktionalität des Prototyps nachzuweisen, sondern auch seine Grenzen sichtbar zu machen und mögliche Ansatzpunkte für spätere Erweiterungen zu identifizieren.

1.5 Aufbau der Arbeit

Nach der Einleitung werden zunächst die für das Verständnis der Arbeit erforderlichen fachlichen und technischen Grundlagen dargestellt. Anschließend folgt die Analyse des Problemraums, der Anforderungen sowie relevanter bestehender Ansätze.

Darauf aufbauend beschreibt die Arbeit die Konzeption des entwickelten Systems und erläutert anschließend dessen prototypische Implementierung. Abschließend erfolgt eine Evaluation der Lösung, bevor die zentralen Ergebnisse zusammengefasst und mögliche Erweiterungen im Ausblick dargestellt werden.

2 Grundlage

2.1 Grundbegriffe und Definitionen

Die folgenden Ausführungen führen die zentralen Begriffe ein, die für das Verständnis der im Rahmen dieser Arbeit entwickelten Monitoring-Lösung erforderlich sind. Sie schaffen ein einheitliches Begriffsverständnis als Grundlage für die nachfolgenden Kapitel zur Analyse, Konzeption und Implementierung.

2.1.1 Netzwerk-Monitoring, Verfügbarkeit und Latenz

Unter Netzwerk-Monitoring wird die kontinuierliche Überwachung des Zustands und der Leistungsfähigkeit von Netzwerkkomponenten, Diensten und Schnittstellen verstanden. Dazu gehören beispielsweise Server, Kommunikationsdienste und Webanwendungen, die über ein Netzwerk erreichbar sein müssen. Monitoring-Systeme erfassen hierzu Daten über Verfügbarkeit, Antwortzeiten und Fehlersituationen, um Störungen frühzeitig zu erkennen und geeignete Maßnahmen einleiten zu können [1].

Die Verfügbarkeit eines Dienstes beschreibt, in welchem Anteil einer betrachteten Zeitspanne dieser seinen vorgesehenen Zweck erfüllen kann. Sie wird häufig als Prozentwert angegeben, beispielsweise als „99,9 % Verfügbarkeit“. Für die operative Überwachung ist zusätzlich die Erreichbarkeit zu einem konkreten Zeitpunkt relevant. Ein Ziel gilt als erreichbar, wenn eine Verbindung innerhalb eines vorgegebenen Zeitlimits aufgebaut werden kann. Schlägt der Verbindungsaufbau fehl oder wird das Zeitlimit überschritten, wird das Ziel als nicht erreichbar bewertet.

Neben der Erreichbarkeit ist die Latenz eine wichtige Kennzahl zur Beurteilung der Dienstqualität. Sie beschreibt die Zeitspanne zwischen dem Start einer Anfrage und dem Eintreffen einer zugehörigen Antwort beziehungsweise eines Verbindungsstatus. Im Netzwerkumfeld wird sie üblicherweise in Millisekunden angegeben. Eine erhöhte Latenz kann auf Überlastung, Netzwerkprobleme oder eine eingeschränkte Leistungsfähigkeit eines Dienstes hinweisen.

2.1.2 Metrikbasiertes Monitoring und Alerting

Eine Metrik ist eine messbare Kennzahl, die einen bestimmten Aspekt des Zustands oder der Leistungsfähigkeit eines Systems beschreibt. Beispiele sind CPU-Auslastung, Arbeitsspeicherkonsum, Anzahl von Anfragen oder Latenzwerte. Bei einem metrikbasierten Monitoring werden diese Kennzahlen in regelmäßigen Abständen erfasst und als Zeitreihen gespeichert. Dadurch können aktuelle Zustände bewertet sowie Veränderungen über einen längeren Zeitraum analysiert werden [1].

Ein Exporter stellt Metriken in einem standardisierten Format über einen Endpunkt bereit. Ein Monitoring-System kann diese Daten anschließend abholen und speichern. Prometheus verwendet hierfür ein Pull- bzw. Scraping-Modell. Dabei ruft der Prometheus-Server die Metrik-Endpunkte der konfigurierten Ziele in festgelegten Intervallen ab [8].

Alerting bezeichnet die automatische Erkennung und Meldung kritischer Zustände auf Grundlage von Metriken. Eine Alert-Regel definiert eine Bedingung, bei deren Erfüllung eine Meldung erzeugt wird. Beispiele sind die Nichterreichbarkeit eines Dienstes oder

das Überschreiten eines Latenzgrenzwertes. Durch Alerting können Verantwortliche über relevante Störungen informiert werden, ohne ein Dashboard kontinuierlich überwachen zu müssen [6].

2.1.3 Containerisierung und TCP-basierte Erreichbarkeitsprüfung

Container sind isolierte Laufzeitumgebungen, in denen Anwendungen zusammen mit ihren Abhängigkeiten ausgeführt werden. Im Vergleich zu virtuellen Maschinen teilen Container den Kernel des Host-Betriebssystems und können daher mit geringerem Ressourcenaufwand gestartet und betrieben werden. Ein Container-Image dient dabei als Vorlage, aus der ein oder mehrere gleichartige Container erzeugt werden können.

Die gemeinsame Verwaltung mehrerer Container wird als Container-Orchestrierung bezeichnet. Sie umfasst unter anderem die Konfiguration von Diensten, Netzwerken, Volumes und Abhängigkeiten. Docker Compose ermöglicht die deklarative Definition und Ausführung von Anwendungen, die aus mehreren Containern bestehen, mithilfe einer YAML-Datei [2].

Die TCP-basierte Erreichbarkeitsprüfung dient der Überwachung eines konkreten Dienstes auf einem definierten Host und Port. Das Transmission Control Protocol (TCP) ist verbindungsorientiert und setzt vor einer Datenübertragung einen Verbindungsaufbau zwischen Client und Server voraus. Kann eine TCP-Verbindung erfolgreich aufgebaut werden, ist der Dienst auf dem angegebenen Port grundsätzlich erreichbar. Ein fehlgeschlagener Verbindungsaufbau weist dagegen auf einen nicht verfügbaren Dienst, ein Netzwerkproblem oder eine blockierende Netzwerkkonfiguration hin.

2.2 Technologische Grundlagen

Die entwickelte Monitoring-Lösung basiert auf mehreren miteinander verbundenen Technologien. Spring Boot bildet die Grundlage des Monitoring-Dienstes. Micrometer, Prometheus, Grafana und Alertmanager übernehmen die Erfassung, Speicherung, Visualisierung und Verarbeitung von Monitoring-Daten. Docker und Docker Compose ermöglichen die reproduzierbare Bereitstellung der beteiligten Komponenten.

2.2.1 Spring Boot, Actuator und Micrometer

Spring Boot ist ein Framework zur Entwicklung Java-basierter Anwendungen. Es vereinfacht die Konfiguration typischer Komponenten und unterstützt unter anderem die Bereitstellung von Webschnittstellen und Betriebsfunktionen. Das Actuator-Modul stellt produktionsnahe Funktionen und HTTP-Endpunkte für Zustandsinformationen, Health-Informationen und Metriken bereit [10].

Micrometer ist eine Metrik-Fassade für Java-Anwendungen. Die Bibliothek ermöglicht die Erfassung von Messwerten unabhängig vom konkret eingesetzten Monitoring-Backend. Spring Boot Actuator unterstützt Micrometer durch automatische Konfiguration und kann Metriken für unterschiedliche Monitoring-Systeme bereitstellen [10].

Für Prometheus stellt Spring Boot bei aktivierter Prometheus-Registry den Endpunkt `/actuator/prometheus` bereit. Dieser Endpunkt liefert Metriken im Prometheus-kompatiblen Format und kann regelmäßig durch einen Prometheus-Server abgefragt werden [10]. In der entwickelten Anwendung dient dieser Mechanismus dazu, selbst erzeugte Status- und Latenzmetriken für die weitere Verarbeitung bereitzustellen.

2.2.2 Prometheus, Grafana und Alertmanager

Prometheus ist ein Monitoring-System, das Metriken von konfigurierten Zielen abrufen und als Zeitreihen speichert. Die Konfiguration definiert unter anderem Abrufintervalle, Zielsysteme, Metrikpfade und Alert-Regeln. Auf Grundlage der gespeicherten Zeitreihen können Abfragen durchgeführt, Visualisierungen erstellt und kritische Zustände erkannt werden [8].

Im entwickelten System fragt Prometheus den Actuator-Endpunkt der Spring-Boot-Anwendung in festen Intervallen ab. Die Anwendung stellt dabei unter anderem die Metriken `network_device_up` und `network_device_latency_ms` bereit. Diese Werte werden von Prometheus als Zeitreihen gespeichert und können anschließend durch Grafana oder durch Alert-Regeln ausgewertet werden.

Grafana dient der Visualisierung der in Prometheus gespeicherten Daten. Prometheus wird dabei als Datenquelle konfiguriert. Grafana ermöglicht die Darstellung von Zeitreihen, Tabellen und Statusanzeigen, wodurch beispielsweise Verfügbarkeit und Latenz einzelner Netzwerkziele nachvollziehbar werden [3].

Alertmanager verarbeitet Alerts, die von Prometheus aufgrund definierter Regeln erzeugt werden. Zu seinen zentralen Aufgaben zählen die Deduplizierung, Gruppierung und Weiterleitung von Meldungen an konfigurierte Empfänger [7]. In der vorliegenden Lösung können Regeln beispielsweise einen Alert auslösen, wenn ein Ziel über einen definierten Zeitraum nicht erreichbar ist oder ein festgelegter Latenzgrenzwert überschritten wird.

2.2.3 Containerisierte Bereitstellung mit Docker und Docker Compose

Die Monitoring-Lösung wird mit Docker und Docker Compose als mehrteilige Containeranwendung bereitgestellt. Docker Compose ermöglicht es, Services, Netzwerke und Volumes in einer gemeinsamen YAML-Datei zu beschreiben und als zusammengehörigen Anwendungsstack zu starten [2].

Die Anwendung besteht aus getrennten Containern für den Spring-Boot-Monitoring-Dienst, Prometheus, Grafana und Alertmanager. Ein gemeinsames Docker-Netzwerk ermöglicht die Kommunikation der Dienste über ihre Servicenamen. Beispielsweise kann Prometheus den Monitoring-Dienst über den Hostnamen `app` erreichen, während Grafana Prometheus über den Hostnamen `prometheus` als Datenquelle verwendet.

Für Komponenten mit dauerhaft benötigten Daten, insbesondere Prometheus und Grafana, können Volumes verwendet werden. Damit bleiben Zeitreihendaten und lokale Einstellungen auch nach einem Neustart einzelner Container erhalten. Die containerisierte Bereitstellung reduziert den manuellen Installationsaufwand und erleichtert eine reproduzierbare Ausführung der gesamten Monitoring-Umgebung.

2.3 Verwendete Methoden und Konzepte des entwickelten Systems

Die entwickelte Monitoring-Lösung folgt mehreren methodischen und architektonischen Prinzipien. Im Mittelpunkt stehen eine konfigurationsbasierte Verwaltung der Überwachungsziele, die regelmäßige Überprüfung ihrer Erreichbarkeit sowie die Bereitstellung der Ergebnisse als Metriken. Ergänzend wurden eine klar getrennte Schichtenarchitektur, eine robuste Fehlerbehandlung und automatisierte Tests für zentrale Funktionen umgesetzt.

2.3.1 Konfigurationsgetriebenes Gerätemanagement und zeitgesteuertes Polling

Die zu überwachenden Netzwerkziele werden nicht fest im Quellcode hinterlegt, sondern über die Anwendungskonfiguration definiert. Zu jedem Ziel werden ein Name, ein Host, ein Port sowie ein Aktivierungsstatus angegeben. Zusätzlich werden zentrale Parameter wie das Prüfintervall und das Verbindungs-Timeout konfiguriert.

Die Konfiguration wird beim Start der Anwendung in eine strukturierte Konfigurationsklasse übernommen. Dadurch wird die technische Beschreibung der Überwachungsziele von der eigentlichen Prüfungslogik getrennt. Neue Ziele können durch die Anpassung der Konfigurationsdatei ergänzt werden, ohne dass Änderungen an den Java-Klassen erforderlich sind. Der Aktivierungsstatus ermöglicht außerdem, einzelne Ziele bei Bedarf vorübergehend von der Überwachung auszunehmen.

Die Überwachung erfolgt nach dem Prinzip des zeitgesteuerten Pollings. Dabei überprüft der Monitoring-Dienst alle aktivierten Ziele in regelmäßigen Abständen. Für die Umsetzung wird die Scheduling-Funktion des Spring Framework verwendet. Methoden, die mit der Annotation `@Scheduled` versehen sind, können in festgelegten Zeitintervallen automatisiert ausgeführt werden [11].

Das Polling-Verfahren stellt einen Kompromiss zwischen Aktualität und Ressourcenverbrauch dar. Kürzere Intervalle ermöglichen eine schnellere Erkennung von Störungen, verursachen jedoch mehr Verbindungsversuche und Metrikaktualisierungen. Längere Intervalle reduzieren den Aufwand, führen jedoch zu einer größeren Verzögerung bei der Erkennung eines Ausfalls. Durch die Konfigurierbarkeit des Intervalls kann dieser Kompromiss an die jeweilige Einsatzumgebung angepasst werden.

2.3.2 Zustandsbasierte Metriken und Kennzahlenmodell

Die Ergebnisse der Erreichbarkeitsprüfungen werden als zustandsbasierte Metriken modelliert. Für jedes überwachte Ziel werden zwei Kennzahlen bereitgestellt: ein Verfügbarkeitsstatus und eine Latenzmetrik. Der Status wird durch die Metrik `network_device_up` dargestellt. Ein Wert von 1 bedeutet, dass der Verbindungsaufbau erfolgreich war; ein Wert von 0 kennzeichnet ein nicht erreichbares Ziel.

Die zweite Metrik `network_device_latency_ms` enthält die Dauer des letzten Verbindungsversuchs in Millisekunden. Beide Werte werden als Gauges exportiert. Ein Gauge ist für diese Art von Information geeignet, da er einen aktuellen Zustand beschreibt und sich sowohl erhöhen als auch verringern kann. Dies unterscheidet ihn beispielsweise von einem Counter, der ausschließlich monoton ansteigt [9].

Zur Unterscheidung der einzelnen Ziele werden die Metriken mit den Labels `device` und `host` versehen. Dadurch können Prometheus-Abfragen, Alert-Regeln und Grafana-Dashboards den Status und die Latenz eines konkreten Ziels auswerten. Das Kennzahlenmodell bildet damit die Grundlage für die Visualisierung und die automatische Erkennung von Störungen.

2.3.3 Schichtenarchitektur, Fehlerbehandlung und Robustheit

Die Anwendung ist in mehrere fachlich getrennte Komponenten gegliedert. Diese Struktur folgt dem Prinzip der Schichtenarchitektur und unterstützt eine klare Zuordnung von Verantwortlichkeiten. Die Konfigurationskomponente übernimmt das Einlesen und Bereitstellen der Einstellungen aus der Datei `application.yml`. Das Datenmodell umfasst die Klassen `Device` und `CheckResult`. Während `Device` ein zu überwachendes Ziel

beschreibt, enthält `CheckResult` das Ergebnis einer konkreten Prüfung einschließlich Erreichbarkeitsstatus, Latenz und Prüfzeitpunkt.

Die zentrale fachliche Logik befindet sich im `DeviceCheckService`. Dieser Dienst führt die Prüfungen aus, aktualisiert die Metrikwerte und hält die zuletzt ermittelten Prüfergebnisse bereit. Der `MonitoringScheduler` stößt die periodische Ausführung an. Die Controller-Schicht stellt REST-Endpunkte bereit, über die Geräte, aktuelle Prüfergebnisse und der Status des Monitoring-Dienstes abgefragt werden können. Zusätzlich ist eine manuelle Auslösung eines Prüfzyklus vorgesehen.

Netzwerkprüfungen sind fehleranfällig, da Verbindungsprobleme, nicht verfügbare Dienste, ungültige Zieladressen oder Zeitüberschreitungen auftreten können. Deshalb werden Ausnahmen beim Aufbau einer TCP-Verbindung behandelt. Schlägt ein Verbindungsaufbau fehl, wird das betreffende Ziel als nicht erreichbar eingestuft; die Prüfung der weiteren Ziele wird dennoch fortgesetzt.

Ein konfigurierbares Timeout begrenzt die Wartezeit für jeden Verbindungsversuch. Dadurch wird verhindert, dass nicht reagierende Ziele den Prüfzyklus über einen längeren Zeitraum blockieren. Auch bei einem fehlgeschlagenen Verbindungsversuch werden Status und Dauer der Prüfung als Ergebnis gespeichert. Die Kombination aus Fehlerbehandlung, Timeout und unabhängiger Verarbeitung einzelner Ziele trägt zur Robustheit der Monitoring-Lösung bei.

2.3.4 Automatisierte Tests der Kernlogik

Zur Absicherung zentraler Funktionen wurden automatisierte Unit-Tests mit JUnit 5 umgesetzt. JUnit stellt einen Rahmen zur Definition und automatisierten Ausführung von Tests für Java-Anwendungen bereit [4]. Der Schwerpunkt liegt auf dem `DeviceCheckService`, da dieser die Konfiguration der Ziele verarbeitet und die Ergebnisse der Erreichbarkeitsprüfungen erzeugt.

Ein Test prüft, ob die in der Konfiguration definierten Geräte korrekt in die interne Geräteliste übernommen werden. Dadurch wird sichergestellt, dass die konfigurationsgetriebene Verwaltung der Ziele wie vorgesehen funktioniert. Ein weiterer Test verwendet ein nicht erreichbares Ziel und prüft, ob dieses als nicht erreichbar bewertet wird. Zusätzlich wird geprüft, ob für den Verbindungsversuch ein gültiger Latenzwert erzeugt wird.

Die Tests ermöglichen eine wiederholbare Überprüfung der Kernlogik, ohne dass dafür die vollständige Containerumgebung mit Prometheus, Grafana und Alertmanager gestartet werden muss. Sie reduzieren damit das Risiko, dass Änderungen an der Anwendung unbemerkt zu Fehlern in der grundlegenden Monitoring-Funktion führen.

2.4 Stand der Technik und Einordnung

Blackbox Exporter???

3 Analyse

4 Konzeption

5 Implementierung

6 Evaluation

7 Zusammenfassung und Ausblick

Literatur

- [1] Brian Brazil. *Prometheus: Up & Running*. O'Reilly, 2019.
- [2] Docker Inc. *Docker Compose*. <https://docs.docker.com/compose/> (Zugriff am 16.08.2026). 2026.
- [3] Grafana Labs. *Configure the Prometheus Data Source*. <https://grafana.com/docs/grafana/latest/datasources/prometheus/configure/> (Zugriff am 16.08.2026). 2026.
- [4] JUnit Team. *JUnit 5 User Guide*. <https://junit.org/junit5/docs/current/user-guide/> (Zugriff am 16.08.2026). 2026.
- [5] Donald E. Knuth. „Computer Programming as an Art“. In: *Communications of the ACM* 17.12 (1974), S. 667–673.
- [6] Prometheus Authors. *Alerting Overview*. <https://prometheus.io/docs/alerting/latest/overview/> (Zugriff am 16.08.2026). 2026.
- [7] Prometheus Authors. *Alertmanager*. <https://prometheus.io/docs/alerting/latest/alertmanager/> (Zugriff am 16.08.2026). 2026.
- [8] Prometheus Authors. *Configuration*. <https://prometheus.io/docs/prometheus/latest/configuration/configuration/> (Zugriff am 16.08.2026). 2026.
- [9] Prometheus Authors. *Metric Types*. https://prometheus.io/docs/concepts/metric_types/ (Zugriff am 16.08.2026). 2026.
- [10] Spring Team. *Metrics :: Spring Boot*. <https://docs.spring.io/spring-boot/reference/actuator/metrics.html> (Zugriff am 16.08.2026). 2026.
- [11] Spring Team. *Task Execution and Scheduling*. <https://docs.spring.io/spring-framework/reference/integration/scheduling.html> (Zugriff am 16.08.2026). 2026.

Abbildungsverzeichnis

Tabellenverzeichnis

Listings

Abkürzungsverzeichnis

Anhang

A Erster Abschnitt des Anhangs

In den Anhang gehören „Hintergrundinformationen“, also weiterführende Information, ausführliche Listings, Graphen, Diagramme oder Tabellen, die den Haupttext mit detaillierten Informationen ergänzen.

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Kolophon

Dieses Dokument wurde mit der L^AT_EX-Vorlage für Abschlussarbeiten an der htw saar im Bereich Informatik/Mechatronik-Sensortechnik erstellt (Version 2.25, August 2024). Die Vorlage wurde von Yves Hary und André Miede entwickelt (mit freundlicher Unterstützung von Thomas Kretschmer, Helmut G. Folz und Martina Lehser). Daten: (F)10.95 – (B)426.79135pt – (H)688.5567pt